

OSTRICH2: Solver for Complex String Constraints

Matthew Hague 
 Department of Computer Science
 Royal Holloway, University of London
 matthew.hague@rhul.ac.uk

Denghang Hu 
 Key Laboratory of System Software
 Institute of Software
 Chinese Academy of Sciences
 hudh@ios.ac.cn

Artur Jeż 
 Institute of Computer Science
 University of Wrocław
 aje@cs.uni.wroc.pl

Anthony W. Lin 
 University of Kaiserslautern-Landau
 Max-Planck Institute for Software Systems
 awlin@mpi-sws.org

Oliver Markgraf 
 University of Kaiserslautern-Landau
 markgraf@cs.uni-kl.de

Philipp Rümmer 
 University of Regensburg
 Uppsala University
 philipp.ruemmer@ur.de

Zhilin Wu 
 Key Laboratory of System Software
 Institute of Software
 Chinese Academy of Sciences
 wuzl@ios.ac.cn

Abstract—We present OSTRICH2, the latest evolution of the SMT solver OSTRICH for string constraints. OSTRICH2 supports a wide range of complex functions on strings and provides completeness guarantees for a substantial fragment of string constraints, including the straight-line fragment and the chain-free fragment. OSTRICH2 provides full support for the SMT-LIB theory of Unicode strings, extending the standard with several unique features not found in other solvers: among others, parsing of ECMAScript regular expressions (including look-around assertions and capture groups) and handling of user-defined string transducers. We empirically demonstrate that OSTRICH2 is competitive to other string solvers on SMT-COMP benchmarks.

I. Introduction

Strings are ubiquitous in modern software systems, especially with the advent of programming languages like JavaScript, PHP, and Python. Despite this, string manipulation is well-known to be error-prone and can easily lead to security vulnerabilities including HTML injection (e.g. see [?], [?]). Applications to analysis of security vulnerabilities caused by string manipulation have been one of the main catalysts for the extensive research into SMT over strings spanning across the last twenty years [?], [?], [?], [?], [?], [?], [?]. One of the success stories of string solvers includes their usage at AWS for analysis of Role-Based Access Control (RBAC) policies (e.g. see [?], [?]). Since 2020 SMT-LIB Unicode String theory [?] has been formalized and supported by many existing string solvers, including Z3 [?], Z3-alpha [?], Z3-Noodler [?], [?],

[?], [?], Z3str3RE [?], cvc5 [?], Z3str4 [?], Trau [?], [?], [?], and our own solver OSTRICH [?].

In this paper, we present OSTRICH2: the latest evolution of the string solver OSTRICH [?]. The goal behind the development of OSTRICH in 2018 was to address the lack of support for complex string functions like replaceall, transducers, reverse, etc. by string solvers. In particular, these are known to be crucial for capturing real-world string manipulations [?], [?], [?]. In a nutshell, this initial version of OSTRICH employed the so-called backward regular propagation for the so-called straight-line fragments, and leveraged the use of Nielsen’s transformation, when the formula is not straight-line. Over the years, OSTRICH has undergone some substantial changes, which have radically enhanced the solver, resulting (among others) in the superior performance of OSTRICH in competitions (e.g. #1 for QF_S track in SMT-Competition 2023). This paper aims to, for the first time, introduce potential future users to OSTRICH in its latest form. To this end, rather than being exhaustive, we aim for accessibility by taking the reader through illustrative examples (among others). In addition to unravelling the design of OSTRICH2, we also report our latest experimentation of the solver on SMT-COMP’24 and SMT-COMP’25 benchmarks, showing its competitiveness, most notably on unsatisfiable instances.

The following is a list of main features of OSTRICH2:

- (F1) SMT-LIB Unicode String theory inputs.
- (F2) Native support of features of ECMAScript regu-

lar expressions (e.g. lookahead assertions, capture groups, references).

- (F3) Native support of replaceall and, more generally, complex string transductions using a more comprehensive regular constraint propagation strategy, which includes not only backward, but also forward regular propagations (among others).
- (F4) An extensive portfolio of solving strategies that includes a cost-enriched string solver (CE-Str) [?], a list-based ADT solver (ADT-Str), and pre-processings (length/character-count abstraction).

Organization: Section II reviews the SMT-LIB Unicode Strings standard and our grammar (a fully formal description is given in Appendix A); Section III reviews the architecture of OSTRICH2; Section IV details OSTRICH2’s key algorithms; Section V discusses the completeness of the algorithms; Section VI shows that OSTRICH2 is extensible with user-defined string functions; Section VII reports evaluation results; and Section VIII concludes with directions for future work.

II. Specification Language

A. SMT-LIB Standard for Unicode Theory of Strings

We begin with the input language of OSTRICH2. It is based on SMT-LIB 2.6 [?] — in particular, including support for the SMT-LIB Unicode theory and Linear Integer Arithmetic constraints — but additionally also supports some “complex” string functions including transductions. A formal description of the grammar is deferred to Appendix A, and here we illustrate the supported language features by example.

1) SMT-LIB Constraints: Below is a minimal SMT-LIB script to show some core features. The script uses the quantifier-free string theory with linear integer arithmetic (QF_SLIA), contains three string variables x , y , and z , and one integer variable k . It asserts constraints on the variables that are explained below. The final statements check the satisfiability of the constraints and produce a satisfying assignment.

```

1 (set-logic QF_SLIA)
2
3 (declare-fun x () String)
4 (declare-fun y () String)
5 (declare-fun z () String)
6 (declare-fun k () Int)
7
8 (assert (= (str.len x) (+ k 1)))
9 (assert (= x (str.++ y z)))
10 (assert (str.in_re x (re.+ (re.union (str.to_re "a")
    ⇨ (str.to_re "b")))))
11
12 (check-sat)
13 (get-model)

```

Constraints are written in a Lisp-style prefix notation. The first assert requires that the length of x is equal to $k + 1$. The second requires that x is equal to the concatenation of y and z . The third requires that x belongs to the regular language $(\{a\} \cup \{b\})^+$, represents a non-empty sequence of a and b characters.

Standard Boolean connectives are supported. The example below requires x to either be the concatenation of y and z or the concatenation of z and y .

```

1 (assert (or (= x (str.++ y z)) (= x (str.++ z y))))

```

B. Extensions Beyond the SMT-LIB Standard

OSTRICH2 provides additional features to model string constraints in practical applications. A simple example is the `str.reverse` function. Below, we introduce the transducer, regular expression, and automata extensions.

1) Transducer-based Operations: OSTRICH2 supports transducers and prioritised finite-state transducers [?], [?], [?].

A transducer (a.k.a. a rational transducer) takes a single string as input and produces a single output string. They enable operations such as HTML encoding (e.g. replacing `&` with `&`) to be defined. Transducers are written as recursive functions with two arguments: the input and the output. The example below shows a `toUpper` transduction. The base case (line 4) applies when both strings are empty. The recursive case (lines 5–11) asserts three conditions. First, both arguments are non-empty. Second, the head of the output (y) is equal to the upper case version of the head of the input (x). Third, the tails of the two arguments recursively satisfy `toUpper`. To obtain the upper case version of a character, arithmetic is performed on the character codes. In general, a transducer definition can contain multiple mutually recursive functions. The option `:parse-transducers` instructs OSTRICH2 to translate the recursive function definition to an internal transducer representation and apply an automata-based decision procedure to solve the constraints in lines 16–17.

```

1 (set-option :parse-transducers true)
2
3 (define-fun-rec toUpper ((x String) (y String)) Bool
4 (or (and (= x "") (= y ""))
5     (and (not (= x "")) (not (= y ""))
6         (= (char.code (str.head y))
7             (ite (and (<= 97 (char.code (str.head x)))
8                 (<= (char.code (str.head x)) 122))
9                 (- (char.code (str.head x)) 32)
10                (char.code (str.head x))))
11         (toUpper (str.tail x) (str.tail y)))))
12
13 (declare-fun x () String)
14 (declare-fun y () String)
15
16 (assert (= x "Hello World"))
17 (assert (toUpper x y))

```

Prioritised transducers additionally allow certain transitions to take precedence over other transitions. That is, a non-deterministic transduction may only succeed on one path if another cannot succeed. Such transducers are used internally in regular expression handling, described below, but can also be formulated using the `define-fun-rec` notation (not shown here). We discuss these below after we cover extended regular expressions.

2) Extended Regular Expression Support: The SMT-LIB standard defines a minimal set of regular expression constructs, all of which are supported by OSTRICH2. In

addition, OSTRICH2 can also parse ECMAScript regular expressions, following the 2020 version [?], and supports regular expression features such as look-arounds that cannot directly expressed in SMT-LIB. A simple usage of ECMAScript regular expressions is shown below, using the `re.from_ecma2020` function. The expression matches either sequences of characters or sequences of digits.

```
1 (assert (str.in_re w (re.from_ecma2020
    ↪ "[a-z]*|[0-9]*")))
```

To enable full support of ECMAScript character escaping, OSTRICH2 also supports single-quoted strings, in which the normal SMT-LIB character escaping is disabled and strings are passed unmodified to the regular expression parser:

```
1 (assert (str.in_re w (re.from_ecma2020 '\x24[0-9]+')))
```

One notable extension is the use of capture groups and references in replace and extract functions. The following example shows the replacement of any substring matching `src=.*` with the text appearing between the single quotation marks. For example `<img src='image.png'>` would become `<img image.png>`. We use the `re.from_ecma2020` function for convenience; direct functions for regular expression operators—such as `re.capture`—are also available.

```
1 (assert (= x
2   (str.replace_cg_all
3     y
4     (re.from_ecma2020 "src=('.*')")
5     (_ re.reference 1))))
```

There are several features to note about this example. The first argument of `str.replace_cg_all` is the string to be searched. The second is the regular expression that captures part of the string using the parenthesis notation (...). The final argument is the replacement pattern that can be a concatenation of string literals and references to captured text. In this case, only the contents of the first (and only) capture group are used.

Importantly, when matching `src=('.*)'`, precedence rules must be respected. The `*` operator is greedy in ECMAScript (and many other regular expression languages), which means that it should match as many characters as possible before the remainder of the match begins. This can be a cause of errors when developers write regular expressions and a challenge for symbolic execution tools using a string constraint solver [?]. For example, if `y` contained the value `src='a' src='b'`, then there should only be one match of `src=('.*)'` rather than two. In the match, the captured value is `a' src='b'`, that is, the longest string surrounded by `'` symbols. This semantics is respected by OSTRICH2 through the use of prioritised transducers [?], [?], [?]. In a prioritised transducer, steps of the transduction can be given priority over the alternatives. When matching against `src='a' src='b'` there is a choice whether the match of `(.*)` should stop at the second `'` or continue. Because `*` is greedy, the match of `(.*)` has priority and matching can only stop if continuing would fail. When

replacing the pattern `src=('.*)'` with `src=('.*?')`, using a lazy quantifier `*?`, instead the shortest string will be matched.

3) Automata Representations: OSTRICH2 also directly supports finite-state automata, which are often more convenient than regular expressions when integrating an SMT solver in applications. The function `re.from_automaton` parses a finite-state automaton given as a string. In the example below the automaton has

- an initial state `s0`,
- an accepting state `s1`,
- a transition `s0 -> s1 [0,100]`; (accepting character codes 0–100),
- a loop `s1 -> s1 [0,65535]`; (accepting any Unicode character).

Then `str.in_re` can be used to test whether the value of an expression belongs to the language of that automaton.

```
1 (assert (str.in_re x
2   (re.from_automaton "automaton value_0 {init s0;
    ↪ s0 -> s1 [0, 100]; s1 -> s1 [0,65535];
    ↪ accepting s1;}")))
```

III. System Architecture

The overall architecture of OSTRICH2 is depicted schematically in Figure 1. In essence, OSTRICH2 comprises three string solvers that can be used independently or run using a time-sliced portfolio approach.

At its core, OSTRICH2 builds on the SMT solver Princess [?], which provides the logical reasoning framework and support for theories such as linear integer arithmetic. OSTRICH2 provides pre-processing and support for the quantifier-free string theories `QF_S` (pure strings) and `QF_SLIA` (strings with linear integer arithmetic). Princess also provides support for algebraic data-types (ADTs), on top of which the ADT-Str string solver is implemented.

The second and third solver are the Regular Constraint Propagation (RCP) and Cost-Enriched String (CE-Str) engine, respectively. These share similar architectures. They begin with a shared string preprocessor, which processes the SMT-LIB input and forwards it to the SMT Core of Princess. Details of the string preprocessing are given in the next section. The SMT Core coordinates with the LIA Solver for linear integer arithmetic and with a solver engine for string-specific reasoning.

Both solvers uses automata- and transducer-based representations of sets of strings and supported string functions. A main loop applies proof rules like Automata Intersection, Forward (FWD) and Backward (BWD) Propagation, Nielsen Splitting, and other string-related inference rules. Details of the rules are given in the next section. A string database and an automata database are used for efficiently storing and retrieving string and automata data. The automata database is based on the BRICS Automata Library [?], which provides efficient automaton operations.

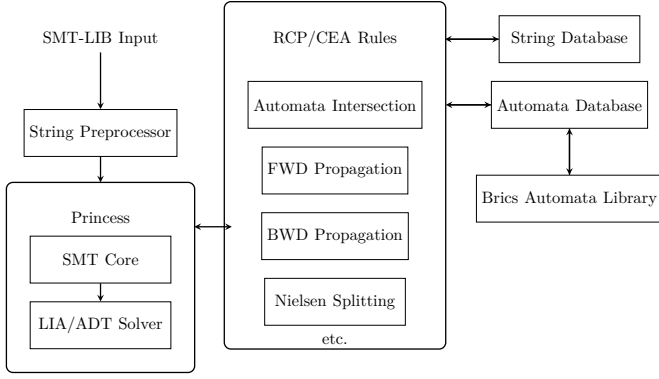


Figure 1. Overall architecture of OSTRICH2: the SMT-LIB input is handled by our string preprocessor and Princess SMT core (with LIA and ADT-Str), while the RCP and CE-Str solvers consist of inference rules that are repeatedly applied using string and automata databases.

For the purposes of presentation, the solvers can be considered to receive and handle constraints from the SMT Core in the following normal form

$$\begin{aligned}
 S &::= A \mid \neg A \mid S \wedge S \\
 A &::= f(x_1, \dots, x_n) \mid x = g(x_1, \dots, x_n) \mid \\
 &\quad x \in L \mid x = \sum_i d_i x_i .
 \end{aligned}$$

Here $x, x_1, \dots, x_n$ are string or integer variables and d_i are integer constants, f denotes a string predicate over parameters $x_1, \dots, x_n$, and g denotes a string function with input parameters $x_1, \dots, x_n$ and output x . We will make use of standard string functions and predicates such as `concat`, `prefixof`, `contains`, and `suffixof` as needed. We sometimes write $|t|$ to denote the length of the word represented by the term t . For example, the constraint

```

1 (assert (and (= (str.++ x y) (str.++ y x))
2   (str.in_re x (re.from_ecma2020 "a*ba*"))
3   (str.in_re y (re.from_ecma2020 "a*ca*"))))

```

has the following normal form using an additional variable z

$$z = \text{concat}(x, y) \wedge z = \text{concat}(y, x) \wedge x \in a^*ba^* \wedge y \in a^*ca^* .$$

In the sections below, we given an overview of the three main solver engines. These are: ADT-Str (algebraic data-types), RCP (regular constraint propagation), and CE-Str (cost-enriched strings).

A. ADT-Str: List-Based Solver

The ADT-Str solver builds on the decision procedure for algebraic data-types (ADTs) with size catamorphism implemented in Princess [?]. Algebraic data-types are used to represent strings using the standard encoding of lists with `nil` and `cons` constructors. The length of a string is computed using the build-in size function provided by the ADT solver, mapping every constructor term to the number constructor occurrences. Other SMT-LIB functions

on strings, for instance substring, concatenation, etc., are in ADT-Str encoded using uninterpreted functions and axioms capturing the recursive definition of the string functions. Regular expression matching is implemented using Brzozowski derivatives [?].

ADT-Str is useful, in particular, for computing solutions of string constraints that are outside of the fragments for which the other solvers are complete. ADT-Str can easily handle certain functions that are hard for our propagation algorithms. Those functions include, among others, string-to-integer conversion, and functions like `substring` and `indexof` that calculate with integer offsets.

B. RCP: Regular Constraint Propagation

Regular Constraint Propagation (RCP) is the newest algorithm that has been implemented in OSTRICH2, based on a subset of proof rules in our paper [?]. The main goal of RCP is to prove unsatisfiability of the input constraint. The algorithm handles string functions like `concatenation`, `replace`, `replaceall`, and `regular constraints`. Other string functions (e.g. `reverse`, `one-way` and `two-way transducers`) are also permitted. These functions permit either exact pre-image or exact post-image computation of regular constraints or both. In general, these images need not be exact, but must at least overapproximate the true pre/post image.

The main idea behind RCP is to propagate regular constraints of the form $x \in L$ to other string variables through forward and backward propagations using the RCP inference rule described in the next section.

C. CE-Str: Cost-Enriched String Engine

The main algorithm of CE-Str (based on [?]) applies backward regular constraint propagation. It uses cost-enriched finite automata (CEFAs) instead of standard finite automata to represent sets of words. These automata are able to capture numerical information about accepted strings, such as the string length, the number of characters appearing before a transition is fired, and so on. This allows length constraints on strings and functions like `indexof` to be directly supported.

In addition, CE-Str can handle counting operators more efficiently by leveraging the numerical information encoded in CEFAs. For instance, the regex $a^{\{1,1000\}}$, which accepts strings consisting of 1 to 1000 repetitions of the character a , can be represented as a CEFA with just one state and one transition—compared to the 1000 states and 1000 transitions required by a standard finite automaton.

CE-Str supports a wide range of string functions, including `concatenation`, `replace`, `replaceall`, `substring`, `indexof` and `length`. It provides completeness guarantees for the straight-line fragment of these functions. CE-Str complements RCP and ADT-Str. It performs well on string constraints involving integer type but struggles when the constraints are outside its decidable fragment.

IV. String Theory Algorithms

In this section we describe the core inference rules and algorithmic components that OSTRICH2 applies to solve string constraints. We first present the preprocessing steps shared by the RCP and CE-Str solvers. We then present the inference rules applied in the main loops of RCP and CE-Str.

A. Preprocessing

Before applying the core inference rules, OSTRICH2 performs a series of preprocessing steps to enrich the constraint set with auxiliary information and simplify trivial cases. The simplification rules are given below. First we describe the simplification rules used by both RCP and CE-Str, then we describe the additional rules used by RCP only.

1) Common Preprocessing:

Prefix/Contains/Suffix Simplification: We apply the following rewrites when one argument is concrete:

- $\text{prefixof}(s, t)$ / $\text{suffixof}(s, t)$ with concrete s : replace by

$$t \in L(s.*) \quad / \quad t \in L(.s).$$

- $\text{contains}(t, s)$ with concrete s : replace by

$$t \in L(.s.*).$$

In positive contexts we instead encode:

$$\begin{aligned} \text{prefixof}(s, t) &\rightsquigarrow t = \text{concat}(s, u) \\ \text{suffixof}(s, t) &\rightsquigarrow t = \text{concat}(u, s) \end{aligned}$$

for fresh u . Finally, $\text{prefixof}(t, t)$, $\text{contains}(t, t)$, $\text{suffixof}(t, t)$ are trivially true and replaced by the Boolean constant **true**.

Common Sub-Expression Elimination: While OSTRICH2 does not apply aggressive sharing of terms and expressions, it is useful to merge repeated occurrences of certain operators that are relatively expensive to handle, including expressions $\text{indexof}(x, y, i)$. For expressions e of this kind that occur multiple times in a formula, OSTRICH2 introduces a fresh variable k and adds the equation $k = e$. All occurrences of e in the formula are then rewritten as k . This rewriting ensures that the solver only sees one copy of each complex sub-expression and speeds up subsequent reasoning.

2) RCP Preprocessing: The next simplifications are currently used by RCP only.

a) Length and Character-Count Approximation: Infers approximate bounds on string lengths and character counts for complex string functions.

- Derived length constraints: derive length constraints and character count approximations from word equations and other string functions. E.g. $|x| = |y| + |z|$ is added for $x = \text{concat}(y, z)$.
- Automaton-transition analysis: inspect each transition in an automaton A and, for any character a

not appearing on any transition, add the char count constraint $|x_a| = 0$ for every x constrained by A .

- Index-of range constraints: for each occurrence of $\text{indexof}(x, y, i)$ (which returns the first index of y in x after position i), add implied constraints on the result. E.g., $-1 \leq \text{indexof}(x, y, i) \leq |x|$ captures that the result must be either -1 or in the interval $[0, |x|]$.

b) Index-of/Substr/At Rewriting: Translate substr , at , and indexof into equivalent combinations of string concatenation, length constraints, and regex-membership tests by introducing fresh variables and constraints to encode the positional semantics.

We illustrate one such rewrite for $r = \text{substr}(s, i, n)$, which asserts that r contains the longest contiguous substring of s of length at most n starting at position i . It can be split into three cases.

First we introduce fresh string variables p, r, q with

$$s = \text{concat}(p, r, q) \wedge |p| = i,$$

and then distinguish:

- If $i < 0$, $n \leq 0$, or $i \geq |s|$, then $r = \varepsilon$.
- If $0 \leq i < |s|$ and $i + n \leq |s|$, then $|r| = n$, so r is exactly the length- n slice of s starting at position i .
- If $0 \leq i < |s|$ but $i + n > |s|$, then $|r| = |s| - i$, so r is the substring from position i to the end of s .

B. Inprocessing Rules

There is also a set of lightweight simplification rules that are applied to expressions during proving. Such rules are applied in the local context of a proof goal and are often able to significantly simplify expressions, for instance by evaluating function applications with known arguments or discovering obvious contractions. During inprocessing, equations and assignments of values to variables are propagated to other constraints. There are also certain preprocessing rules that are applied again during inprocessing; for instance, $\neg \text{suffixof}(s, t)$ can be turned into a regular expression constraint as soon as one of the arguments s, t is a known string, but has to be kept unchanged before that.

C. Inference Rules

The main loops of the RCP and CE-Str repeatedly apply proof rules until a branch is closed or a model is found. We discuss the most important proof rules in this section.

The RCP algorithm assigns priorities to each possible rule application based on an estimation of the workload (e.g. the size of the involved automata) and selects the rule application with the highest priority first. Fairness is ensured by penalizing newer rule applications, and therefore preferring rule applications that have resided in the waiting queue the longest. Priorities are computed as the weighted sum of several criteria:

- Concrete-argument: any rule whose input or output language is a ground string is given high priority.

- Information-gain penalty (backward only): rules whose input automata are universal (e.g. $x \in \Sigma^*$) yield little new information in backward propagation and given low priority.
- Exactness adjustment (forward only): forward rules for functions without an exact post-image (e.g. replaceall with symbolic patterns) are given low priority.
- Cost-based penalty: a weight proportional to the combined size of the input and result automata is subtracted.

Differences between the two solvers in their application of the rules is included in the descriptions. The proof rules may cause branching in the proof search. If a satisfying assignment is found on a branch, it witnesses that the constraint is satisfiable. The Close rule detects unsatisfiable constraints. If all branches are closed, the constraint is unsatisfiable.

a) **Regex-to-Automata**: OSTRICH2 constructs an automaton A for each regular membership constraint in $x \in L$. The automaton is stored in the automaton database, which detects equivalent automata to avoid duplication. In the remaining rules, we make the automaton explicit by writing $x \in L(A)$. The CE-Str solver creates cost-enriched automata. Initially these automata do not track any costs. Cost tracking is introduced during the RCP rule below.

b) **BreakCyclicEquations**: OSTRICH2 detects strongly connected components in the variable-dependency graph induced by concatenation equations of the form

$$x = \text{concat}(y, z) \quad \text{and} \quad y = \text{concat}(a, x).$$

For each cycle it removes one equation and, for every remaining equation $v = \text{concat}(u, w)$ in that cycle, adds a trivial emptiness constraint on one argument (e.g. $w = \varepsilon$ or $u = \varepsilon$) to break the dependency.

For example, from

$$x = \text{concat}(y, z) \quad \wedge \quad y = \text{concat}(a, x)$$

we might remove the first equation and introduce

$$y = \text{concat}(a, x), \quad z = \varepsilon, \quad a = \varepsilon,$$

yielding an acyclic set of equations. This transformation is sound, because in any finite model of the original cyclic equations at least one concatenation argument must be empty, and complete, since any solution of the resulting acyclic system (with the added emptiness constraints) automatically satisfies the dropped equations.

c) **Equation Decomposition**: OSTRICH2 uses two simple heuristics to simplify word equations. In what follows, z, x_1, x_2, y_1, y_2 are arbitrary string terms (each may be a variable or a constant). First, when one side is a constant string w and the other is a concatenation

$$w = \text{concat}(x_1, x_2),$$

in which the length of x_1 is known, OSTRICH2 matches the first $|x_1|$ characters of w with x_1 and the remaining characters with x_2 . Second, if the same variable appears in two concatenations,

$$z = \text{concat}(x_1, x_2) \quad \text{and} \quad z = \text{concat}(y_1, y_2),$$

then whenever $|x_1| = |y_1|$ we infer $x_1 = y_1$ and $x_2 = y_2$. These heuristics often resolve equations without full case splitting.

d) **Close**: If for some variable x we have constraints

$$x \in L(A_1) \wedge \dots \wedge x \in L(A_k)$$

and $\bigcap_{i=1}^k L(A_i) = \emptyset$, OSTRICH2 derives a contradiction and close the branch.

e) **Intersection**: When the +eager flag is on, OSTRICH2 maintains at most one automaton per variable by replacing $x \in L(A) \wedge x \in L(B)$ with $x \in L(A \cap B)$.

f) **LengthAbstraction**: From length inequalities (e.g. $|x| \leq |y| + 5$), derive lower/upper bounds on $|x|$, then assert $x \in L(A)$ where A accepts any word within the derived length bounds.

g) **RCP (Regular Constraint Propagation)**: Propagate $x_1 \in L_1, \dots, x_n \in L_n$ forwards through string functions $x = f(x_1, \dots, x_n)$, or $x \in L$ backwards through f .

For example, on $z = \text{concat}(x, y) \wedge z = \text{concat}(y, x) \wedge x \in a^*ba^* \wedge y \in a^*ca^*$ we can propagate $x \in a^*ba^*$ and $y \in a^*ca^*$ forwards (from input to output) through the string concatenation function concat in $z = \text{concat}(x, y)$ to obtain $z \in a^*ba^*a^*ca^*$. Similarly, we can propagate forwards through $z = \text{concat}(y, x)$ and derive $z \in a^*ca^*a^*ba^*$. Since there are no words matching both $a^*ba^*a^*ca^*$ and $a^*ca^*a^*ba^*$ we can conclude—using the Close rule—that the constraint is unsatisfiable.

Backwards propagation may result in branching. For example, if $x = \text{concat}(y, z)$ and $x = ab$, then either $y = ab \wedge z = \varepsilon$ or $y = a \wedge z = b$ or $y = \varepsilon \wedge z = ab$.

When the variable constraints are given by standard finite automata, a rich selection of string functions support exact forwards and backwards propagation. These include concat , reverse , and replace and replaceall , where the replacement pattern can include string variables or references to capture groups in the search pattern. In general, any function can be supported using over-approximations of the pre- and post-images, but without completeness guarantees.

The CE-Str solver only applies backwards propagation and represents pre-images using cost-enriched automata. This allows functions such as indexof and length to be supported (by introducing costs to the automata), but restricts, for example, which versions of replace and replaceall can be analysed precisely.

h) **NielsenSplitter**: OSTRICH2 invokes Nielsen's transformation $[?]$, $[?]$ on any pair of equations in our normal form

$$z = \text{concat}(x_1, x_2) \quad \text{and} \quad z = \text{concat}(y_1, y_2),$$

where each of z, x_i, y_i may be a variable or a constant. We introduce a fresh string variable w and split the proof into two prefix-alignment cases, guided by current length information in a similar way as was done in Norn [?]. That is

$$|x_1| \geq |y_1|, \quad x_1 = \text{concat}(y_1, w), \quad \text{concat}(w, x_2) = y_2$$

or

$$|y_1| \geq |x_1|, \quad y_1 = \text{concat}(x_1, w), \quad \text{concat}(w, y_2) = x_2.$$

In each branch we align the longer prefix against the shorter one, adding both the corresponding concatenation equalities and the derived length equality (e.g. $|x_1| = |y_1| + |w|$ in the first case).

i) String-Integer-Conversions: For handling expressions $\text{str_to_int}(s) = n$ or $\text{int_to_str}(n) = s$, OSTRICH2 systematically explores the possible values of n . As soon as n has a concrete value, inprocessing rules (Section IV-B) apply that replace the function application with an equation or a regular expression constraint describing the possible values of s . To make this exploration perform well in practical cases, the LIA solver utilized in OSTRICH2 applies interval constraint propagation to narrow down the range of possible values of n as much as possible [?]. The interval for n is then sub-divided, until eventually only one possible value for n remains. This search will not terminate in general, since the domain of n can be infinite, but it tends to derive solutions or contradictions quickly in many practical cases.

j) Index Computation: When handling $\text{indexof}(s, p, k)$ on two concrete strings s, p , OSTRICH2 systematically explores possible values of k , taking into account the fact that any value k not satisfying $0 \leq k \leq |s| - |p|$ will necessarily lead to the result -1 . In the “in-range” branch OSTRICH2 applies interval constraint propagation to determine the possible values of k , sub-dividing the interval of values until a concrete value k has been chosen. As soon as all arguments of indexof have concrete values, inprocessing rules (Section IV-B) can evaluate the function.

k) Cut Rule: When all other rules have been exhausted, OSTRICH2 applies cuts to introduce a candidate solution for a string variable x . For this, OSTRICH2 collects every regular-language membership constraint $x \in L_i$ together with any length bounds $\ell \leq |x| \leq u$. From the intersection of the L_i (and respecting ℓ, u), OSTRICH2 extracts a single accepted word w via a standard automaton search. We then split on the two exhaustive cases

$$x = w \quad \text{vs.} \quad x \notin \{w\}.$$

This rule is always sound because any satisfying assignment for x either equals the chosen w or lies outside $\{w\}$ and guarantees that each string variable will eventually be grounded to a concrete value (or shown impossible).

V. Completeness Results

The RCP solver is (in principle) complete for the straight-line fragment [?] and chain-free fragment [?] of string constraints, in the sense that the underlying proof system can show unsatisfiability of unsatisfiable instances from these fragments, when applying the rules in an appropriate (easy) order. However, the usage of priorities means the rules may be applied in a different order, causing the proof to fail. In practice, failures for these fragments are rare.

In the straight-line fragment we require that the constraints are in normal form as in Section IV-C, and variables can be ordered $x_1, \dots, x_n$ such that for x_i there is exactly one equational constraint $x_i = f_i(x_1, \dots, x_{i-1})$, where f_i is a string function such that the pre-image of a regular set is regular. The definition of the chain-free fragment is a bit involved, in essence it relaxes the assumption on the form of the equation and the number of equations, but still requires that there are no “cycles” of dependencies. For the straight-line fragment, we can propagate backwards and the instance is unsatisfiable if and only if all branches are closed using Close rule. For the chain-free fragment [?] we can adapt the original proof (which works for a different solver) to the rules used in OSTRICH2.

The CE-Str solver also provides a completeness guarantee for a version of the straight-line fragment that includes string constraints and linear integer arithmetic [?]. The supported string functions are those where the pre-image of cost-enriched automata constraints can be expressed using cost-enriched automata. This includes indexof and length as well as some versions of replace and replaceall .

VI. Extensibility

An important feature of OSTRICH is the possibility to extend the solver with minimal effort. OSTRICH’s PreOp trait lets users add new string functions by overriding three core methods. This can be done in either Scala (the main language used to implement OSTRICH) or in Java. Below we show each method in turn, together with the ReversePreOp implementation¹.

First, the method `apply` computes the pre-image of a regular language under f , which is the core operation needed for backward propagation in RCP. The method is passed any existing automata constraints on the r arguments (which may be ignored) and the automaton $\mathcal{A}$ representing the result language. The method returns an iterator over tuples of automata whose images under f lie inside $\mathcal{A}$, plus (optionally) the subset of the input constraints actually used.

```
1 override def apply(
2   argCs: Seq[Seq[Automaton]],
3   resultC: Automaton
```

¹This snippet is abbreviated for clarity. The full implementation in `ReversePreOp.scala` includes additional automaton-type matching and error handling.

```

4 ) : (Iterator[Seq[Automaton]], Seq[Seq[Automaton]]) = {
5   (Iterator(Seq(ReverseAutomaton(resultC))), List())
6 }

```

Second, the method `eval` performs concrete evaluation on ground strings: when all arguments are known, it simply returns the result of applying f .

```

1 override def eval(
2   args: Seq[Seq[Int]]
3 ): Option[Seq[Int]] = Some(args.head.reverse)

```

Third, `forwardApprox` computes a (sound) post-image of input languages under f , used for forward propagation. While returning the universal automaton is always correct, a tighter approximation greatly improves performance. For `str.reverse`, one can exactly compute this by intersecting any input automata and then reversing the resulting automaton.

```

1 override def forwardApprox(
2   argCs: Seq[Seq[Automaton]]
3 ): Automaton = {
4   val prod = ProductAutomaton(argCs)
5   ReverseAutomaton(prod)
6 }

```

Finally, the new string function has to be registered in the core string theory class of OSTRICH. In the file `OstrichStringTheory.scala`, we register `str.reverse` by first declaring its SMT-side function symbol:

```

1 val str_reverse = MonoSortedIFunction(
2   "str.reverse",
3   List(StringSort), StringSort, true, false
4 )

```

We then add it to the `extraStringFunctions` list, pairing the name, the `IFunction`, our `ReversePreOp` implementation, and the simple lambdas that pick out the input and output terms:

```

1 val extraStringFunctions = List(
2   ("str.reverse", str_reverse, ReversePreOp, a =>
3     List(a(0)), a => a(1)))

```

With these two lines in place, any occurrence of `str.reverse` in the SMT-LIB script is recognized by OSTRICH2 and automatically dispatched to our `ReversePreOp` for pre-image computation, concrete evaluation, and forward-approximation.

VII. Experiments

A. Benchmark suites and experimental setup

We evaluate our solver on a representative selection of SMT-COMP benchmarks, including all instances from SMT-COMP 2024² and the full set of problems accepted for SMT-COMP 2025³. From the combined pool of roughly 100 000 QF_S and QF_SLIA problems, we randomly sample 2 000 instances proportional to each track’s share of the total, so that our test set reflects the underlying distribution of problem types. All experiments

²<https://zenodo.org/communities/smt-lib/>

³(available at <https://github.com/SMT-LIB/benchmark-submission>)

were run on a 24×3.1 GHz-core server with 185 GB RAM, invoking each solver in a single thread with a 60s timeout and no memory limit.

B. Performance evaluation

The experiments are conducted on the following solvers: `cvc5` [?], `Z3` [?], `Z3-alpha` [?], `Z3-Noodler` [?], `OSTRICH1` [?], and different engines of OSTRICH2⁴ and OSTRICH2 running the engines in a time-sliced portfolio using the flag `-portfolio=strings`. For the RCP engine we evaluate two complementary configurations: one mode that combines forward and backward propagation without Nielsen splitting (using the flags `+F+B-N`), and another mode (using the flags `-F+B+N`) that disables forward propagation in order to emphasize Nielsen’s equation decomposition. In practice, combining forward propagation and splitting yields limited synergy: the extra word equations generated by Nielsen splitting create significant propagation overhead, so omitting forward propagation often improves overall solving performance.

	CVC5	Z3	Z3-alpha	Z3-Noodler	OSTRICH1
sat	1135	1074	1077	1192	755
unsat	722	726	701	744	645
unknown/timeout	144	200	222	64	600
Solved	1856	1800	1778	1936	1400

Figure 2. The experiment result of `cvc5`, `Z3`, `Z3-alpha`, `Z3-Noodler` and `OSTRICH1` on the SMT-COMP’24/25 benchmarks.

	CVC5	OSTRICH	Z3
sat	0	29	0
unsat	9	31	7
unknown	45	0	53
timeout	6	0	0
solved	9	60	7

Figure 3. The experiment result of different engines of OSTRICH2 on the SMT-COMP’24/25 benchmarks. The flags `+F+B-N` enable forwards/backwards propagation and disable the `NielsenSplitter`. The flags `-F+B+N` disable forwards propagation and enable backwards propagation and the `NielsenSplitter`.

Figure 2 shows that among the off-the-shelf solvers, `Z3-Noodler` leads with 1936 solved problems, followed by `cvc5` (1856), `Z3` (1800), `Z3-alpha` (1778), and `OSTRICH1` (1400). Figure 3 shows that both RCP configurations (1759 and 1769 solves) and CE-Str (1638) represent clear upgrades over `OSTRICH1`, whereas the `ADT-Str`

⁴<https://zenodo.org/records/15378807>

engine (1 156) trails—largely due to its simple algorithm. Importantly, when we combine all four engines in a time-sliced portfolio, OSTRICH2 solves 1 937 problems, putting it on par with Z3-Noodler and ahead of the rest of the field. We further observe that Z3-Noodler excels on the sat instances, while OSTRICH2 performs particularly well on the unsat cases.

Finally, although Z3-Noodler does not support `replace_all`, only 2.4% of the SMT-COMP benchmarks use this string function, so its absence has only a modest effect on the overall rankings. We remark that, since our evaluation uses a stratified random sample of 2 000 instances drawn from the full pool of roughly 100 000 QF_S and QF_SLIA benchmarks, there is inevitably some statistical variance in the exact per-solver ranking; nonetheless, the fact that OSTRICH2 solves essentially as many problems as the leader demonstrates its overall competitiveness.

VIII. Conclusion

This paper has strived to provide a concise introduction of the string solver OSTRICH2, which inputs constraints in an extension of the SMT-LIB theory of Unicode Strings with transducers (through mutual recursion) and ECMAScript regular expressions, among others. We have also empirically demonstrated its competitiveness with other string solvers on SMT-COMP benchmarks, particularly on unsatisfiable instances. Interested readers, who are interested in participating in the development and/or applications of OSTRICH2, are wholeheartedly encouraged to get in touch with us.

Appendix

A. SMT-LIB Standard for Unicode Theory of Strings

We begin by outlining the basic syntactic elements for our formulas and the key string and regular expression operations as defined in the SMT-LIB Unicode theory.

Grammar

We define the grammar as follows:

Formulas:

$$\begin{aligned}
\phi &::= \neg \phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \textit{Atom} \\
\textit{Atom} &::= t_s \sim t_s \mid t_{ar} \sim t_{ar} \mid t_s \in t_{re} \mid \textit{StrPred} \\
\textit{StrPred} &::= \textit{prefixof}(t_s, t_s) \mid \textit{suffixof}(t_s, t_s) \mid \\
&\quad \textit{contains}(t_s, t_s) \\
t_s &::= \textit{BaseStr} \mid \textit{StrPos} \mid \textit{StrRep} \\
\textit{BaseStr} &::= c_{str} \mid x_{str} \mid \textit{concat}(t_s, t_s) \\
\textit{StrPos} &::= \textit{at}(t_s, t_{ar}) \mid \textit{substr}(t_s, t_{ar}, t_{ar}) \\
\textit{StrRep} &::= \textit{rep}(t_s, t_s, t_s) \mid \textit{rep_all}(t_s, t_s, t_s) \mid \\
&\quad \textit{rep_re}(t_s, t_{re}, t_s) \mid \textit{rep_re_all}(t_s, t_{re}, t_s) \\
t_{ar} &::= c_{int} \mid x_{int} \mid t_{ar} + t_{ar} \mid |t_s| \mid \\
&\quad \textit{indexof}(t_s, t_s, t_{ar}) \\
t_{re} &::= \emptyset \mid \Sigma \mid \Sigma^* \mid \textit{toRE}(t_s) \mid t_{re} \cdot t_{re} \mid \\
&\quad t_{re} \cup t_{re} \mid t_{re} \cap t_{re} \mid t_{re}^*
\end{aligned}$$

Explanation

- Formulas (ϕ) and Atoms: The Boolean formulas ϕ are constructed using the standard logical connectives (negation $\neg$, conjunction $\wedge$, and disjunction $\vee$) along with atomic formulas. The atomic formulas include comparisons on string terms t_s (via a placeholder relation $\sim$), arithmetic terms t_{ar} (also using $\sim$), the membership test $t_s \in t_{re}$, and the dedicated string predicates defined by *StrPred*.
- String Predicates (*StrPred*): This subclass of atoms is dedicated to predicates that operate specifically on strings. It includes:
 - *prefixof*(t_s, t_s): Checks if the first string is a prefix of the second.
 - *suffixof*(t_s, t_s): Checks if the first string is a suffix of the second.
 - *contains*(t_s, t_s): Checks if the first string is contained within the second.
- String Terms (t_s): The nonterminal t_s is divided into three subclasses:
 - *BaseStr*: Represents the basic string values. This includes string constants c_s , string variables x_s , or the concatenation of two string terms using the operator *concat*.
 - *StrPos*: Represents string functions that require an integer parameter. These include:
 - * *at*(t_s, t_{ar}): Returns the character (as a string) at a specified position.
 - * *substr*(t_s, t_{ar}, t_{ar}): Returns a substring starting at a given position with a specified length.
 - *StrRep*: Contains the string replacement functions, abbreviated here for conciseness. They include:
 - * *rep*(t_s, t_s, t_s): Literal replacement.

- * $\text{rep_all}(t_s, t_s, t_s)$: Replace all occurrences (literal).
- * $\text{rep_re}(t_s, t_{re}, t_s)$: Regex-based replacement.
- * $\text{rep_re_all}(t_s, t_{re}, t_s)$: Replace all occurrences based on a regex.
- Arithmetic Terms (t_{ar}): The arithmetic expressions include integer constants c_{int} , integer variables x_{int} , the sum of two arithmetic expressions $t_{ar} + t_{ar}$, the length function $|t_s|$ (which returns the length of a string), and the function $\text{indexof}(t_s, t_s, t_{ar})$, which returns the index of one string within another as an integer.
- Regular Expressions (t_{re}): Regular expressions are defined in a manner consistent with standard mathematical notation:
 - $\emptyset$ denotes the empty language.
 - Σ is the alphabet, and Σ^* represents its Kleene closure.
 - $\text{toRE}(t_s)$ converts a string term into its corresponding regular expression.
 - The operators $\cdot$ (concatenation), $\cup$ (union), $\cap$ (intersection), and $*$ (Kleene star) are used to build more complex regular expressions.

This grammar provides a foundation that captures the core constructs of the SMT-LIB Unicode theory for strings.